

IMPORTANT NOTE

- Some scanned figures/tables are partially unreadable in the source PDF.
- For those items, the standard interpretation is used and clearly stated.

=====

SECTION A SOLUTIONS

=====

A. 2024 EXAM - SECTION A

Q1(a) Process state diagram and transitions

Step-by-step:

- 1) New: process is being created.
- 2) Ready: process waits in ready queue for CPU.
- 3) Running: process is currently executing on CPU.
- 4) Waiting/Blocked: process waits for I/O or event.
- 5) Terminated: process finishes execution.

Main transitions:

- New -> Ready (admitted)
- Ready -> Running (scheduler dispatch)
- Running -> Ready (time slice over / preemption)
- Running -> Waiting (I/O/event wait)
- Waiting -> Ready (I/O/event completes)
- Running -> Terminated (exit)

Q1(b) Tree, acyclic-graph, and general graph directory structures

Step-by-step:

- 1) Tree structure: each file/dir has exactly one parent.
 - Advantage: simple, no cycles, easy traversal.
 - Limitation: no sharing of same file under different parents.
- 2) Acyclic graph structure: allows shared files/subdirectories via links but no cycles.
 - Advantage: controlled sharing + no infinite traversal loops.
 - Limitation: reference counting/garbage collection needed.
- 3) General graph structure: cycles allowed.
 - Advantage: maximum flexibility.
 - Limitation: traversal may loop forever unless cycle detection is used.

Q1(c) IPC mechanisms comparison

Step-by-step comparison:

- 1) Shared memory: fastest (no kernel copy after setup), needs synchronization.
- 2) Message passing: safer isolation, easier in distributed systems, slower than shared memory
- 3) Pipes/FIFOs: simple producer-consumer communication.
- 4) Sockets: network-capable IPC between local/remote processes.
- 5) Signals: lightweight event notification, limited data payload.

Q2(a) CPU scheduling performance criteria

- 1) CPU utilization (higher is better)
- 2) Throughput (higher is better)
- 3) Turnaround time (lower is better)
- 4) Waiting time (lower is better)
- 5) Response time (lower is better)
- 6) Fairness/starvation avoidance

Q2(a) Effect of Round Robin time quantum

- 1) Very large quantum -> behaves like FCFS (poor response for interactive jobs).
- 2) Very small quantum -> high context-switch overhead.
- 3) Balanced quantum -> good responsiveness with acceptable overhead.

Q2(b) Priority-preemptive and SRTF scheduling (using interpreted table)

Processes:

P1(AT=0,BT=7,Pr=3), P2(2,4,3), P3(4,1,4), P4(3,4,1), P5(3,2,1)

(Assumption: lower priority number = higher priority)

(i) Gantt charts

Priority preemptive:

0-3 P1 | 3-7 P4 | 7-9 P5 | 9-13 P1 | 13-17 P2 | 17-18 P3

SRTF:

0-2 P1 | 2-3 P2 | 3-5 P5 | 5-6 P3 | 6-9 P2 | 9-13 P4 | 13-18 P1

(ii) Average metrics

Priority preemptive:

- Avg Turnaround Time = 10.4
- Avg Waiting Time = 6.8
- Avg Response Time = 5.6

SRTF:

- Avg Turnaround Time = 7.8
- Avg Waiting Time = 4.2
- Avg Response Time = 1.4

Q3(a) Deadlock scenario in RAG

Step-by-step example:

- 1) P1 holds R1 and requests R2.
- 2) P2 holds R2 and requests R3.
- 3) P3 holds R3 and requests R1.
- 4) Circular wait forms: P1->R2->P2->R3->P3->R1->P1.
- 5) No process can proceed => deadlock.

Detection: cycle detection in wait-for graph (single instance resources) or algorithmic detection for multi-instance resources.

Q3(b) Tape-drive state classification

Given:

- Total tapes = 12
- Max needs: P1=10, P2=4, P3=9
- Current allocation: P1=5, P2=2, P3=3 (available=2)
- Remaining need: P1=5, P2=2, P3=6

Check safety:

- 1) Available=2 can satisfy only P2 (need=2).
- 2) After P2 finishes, available becomes 4.
- 3) P1 still needs 5 and P3 needs 6 -> neither can finish.

Conclusion: unsafe state (not necessarily deadlocked yet).

Q3(c)

(i) Additional request decision from given RAG figure:

- Rule: grant only if resulting state remains safe and no deadlock cycle is forced.
- If request creates unsafe/cyclic state, deny/postpone.

(ii) Cycle sufficiency:

- In single-instance resource systems, cycle is both necessary and sufficient.

- In multi-instance systems, cycle is necessary but not always sufficient.

Q4(a) RAID 0/1/4/5 summary

RAID 0: striping, fastest, no fault tolerance.

RAID 1: mirroring, high reliability, 50 percent usable capacity.

RAID 4: block striping + dedicated parity disk, parity bottleneck possible.

RAID 5: block striping + distributed parity, better write balance than RAID 4.

RAID 4 vs RAID 5 parity placement:

- RAID 4: parity on one dedicated disk.
- RAID 5: parity distributed across all disks.

RAID 1 best-case tolerance:

- In best case, one disk per mirror pair can fail without data loss.
- With N mirrored pairs, up to N disks can fail if failures occur in different pairs.

Q4(b) Disk scheduling (queue: 450,200,900,100,500,600; start=550)

(Assumption for directional algorithms: initial direction is upward)

FCFS total movement:

$$= |550-450| + |450-200| + |200-900| + |900-100| + |100-500| + |500-600|$$

$$= 2350$$

SSTF sequence: 550->500->450->600->200->100->900

Total = 1550

LOOK sequence: 550->600->900->500->450->200->100

Total = 1150

C-LOOK sequence: 550->600->900->100->200->450->500

Total = 1550

C-SCAN sequence: 550->600->900->2999->0->100->200->450->500

Total = 5948

----- A. 2021 EXAM - SECTION A

Q1(a) Security in multiprogramming/time-sharing

Two example problems:

- 1) Unauthorized memory/data access by another user process.
- 2) Privilege abuse or account/session hijacking.

Can dedicated-level security be guaranteed in time-sharing?

- Practically no exact equivalence; attack surface is larger in shared systems.
- Strong isolation, access control, auditing, and hardening can still provide very high security.

Q1(b) Cache coherence issues

- 1) Single-processor: stale cached copy after DMA/I/O updates memory.
- 2) Multiprocessor: one core writes, another core reads stale cache line.
- 3) Distributed: replicated data at different nodes diverges without consistency protocol.

Q1(c) Open-source OS and P2P applications

Advantages: transparency, flexibility, community support, lower cost.

Disadvantages: variable support quality, compatibility issues, steeper learning curve.

P2P suitable apps: file sharing, blockchain, distributed backup, collaborative platforms.

Q2(a) Two IPC modes and strengths/weaknesses

1) Shared memory:

- + very fast
- requires explicit synchronization

2) Message passing:

- + simpler protection and modularity
- kernel overhead and generally slower

Q2(b) iOS vs Android

Similarities: mobile OS, app ecosystem, touch UI, networking/sensors/security model.

Differences: ecosystem openness, hardware diversity, customization, app distribution controls

Why Android Java is non-standard:

- Android historically used Dalvik/ART runtime and Android SDK APIs, not standard JVM + desktop Java APIs.

Q2(c) Five OS file-management activities

- 1) Create/delete files
- 2) Create/delete directories
- 3) Provide primitives for file manipulation
- 4) Map files to secondary storage
- 5) Backup/recovery and protection management

Q2(d) Short concepts

- Context switch: saving/restoring process/thread CPU state when switching execution.
- Memory stall: CPU wait time due to slow memory/cache miss.
- Soft deadline: missing deadline degrades quality but is not catastrophic.

Q3(a) CPU scheduling criteria

CPU utilization, throughput, turnaround time, waiting time, response time, fairness.

Q3(b) Scheduling with table from 2021 paper

Processes:

P1(0,7,3), P2(2,4,2), P3(4,1,4), P4(5,4,1), P5(3,2,1)

(i) Gantt charts

Priority preemptive:

0-2 P1 | 2-3 P2 | 3-5 P5 | 5-9 P4 | 9-12 P2 | 12-17 P1 | 17-18 P3

SRTF:

0-2 P1 | 2-3 P2 | 3-5 P5 | 5-6 P3 | 6-9 P2 | 9-13 P4 | 13-18 P1

(ii) If CPU is idle for first 2 units, then SRTF:

Gantt: 0-2 IDLE | 2-3 P2 | 3-5 P5 | 5-6 P3 | 6-9 P2 | 9-13 P4 | 13-20 P1

Turnaround times:

P1=20, P2=7, P3=2, P4=8, P5=2

Average turnaround = $(20+7+2+8+2)/5 = 7.8$

Q3(c) Round Robin quantum effect

- Very large quantum: FCFS-like behavior.
- Very small quantum: high overhead.
- Moderate quantum: good interactive performance.

Q4(a) Deadlock definition and necessary conditions

Deadlock: processes wait forever for resources held by each other.

Necessary conditions:

- 1) Mutual exclusion
- 2) Hold and wait

- 3) No preemption
- 4) Circular wait

Q4(b) Given resource-holding/request state

Method:

- 1) Compute available resources.
- 2) Check if any process can finish.
- 3) Release and repeat.

Based on interpreted data, no unavoidable deadlock is indicated (safe completion order exists)

Q4(c) Banker algorithm state check

- Because the scanned allocation/max table is partially unreadable, exact numeric safe-sequen verification is not fully reliable from OCR text.
- Correct method: Need = Max - Allocation, then repeatedly find process with Need <= Availabl

=====

SECTION B SOLUTIONS

=====

B. 2024 EXAM - SECTION B

Q4(a)

(i) Purpose of MMU:

- Translates virtual/logical addresses to physical addresses.
- Enforces protection and isolation.
- Supports paging/segmentation.

(ii) External vs internal fragmentation:

- External: free memory split into small noncontiguous holes.
- Internal: allocated block has unused internal space.

(iii) TLB effect on EAT:

- Higher TLB hit ratio lowers effective memory access time significantly.

Q5(b) Inverted Page Table

Step-by-step:

- 1) One table entry per physical frame (not per virtual page).
- 2) Each entry stores owning process ID and virtual page number.
- 3) On access, OS/hardware searches (typically hashed) to find matching entry.
- 4) Reduces page-table memory usage for large virtual spaces.

Q5(c) Segment table address translation

Given:

S0: base 520, limit 500

S1: base 350, limit 150

S2: base 100, limit 200

S3: base 1800, limit 600

S4: base 1050, limit 650

Rule: physical = base + offset if offset < limit; else invalid.

- (i) (2,50) -> 100 + 50 = 150 (valid)
- (ii) (4,700)-> invalid (700 >= 650)
- (iii) (0,400)-> 520 + 400 = 920 (valid)
- (iv) (1,100)-> 350 + 100 = 450 (valid)
- (v) (3,500)-> 1800 + 500 = 2300 (valid)

Q6(a) Page-fault occurrence and handling

When page not present in RAM is referenced.

Handling steps:

- 1) Trap to OS page-fault handler.
- 2) Validate reference (legal/illegal).
- 3) Locate page on disk.
- 4) Select free frame or victim (replacement).
- 5) If victim dirty, write back.
- 6) Read required page into frame.
- 7) Update page table/TLB.
- 8) Restart faulting instruction.

Q6(b) LRU page faults (4 frames)

Reference string interpreted from scan:

1,2,1,2,3,4,5,3,4,1,6,7,8,7,9,7,8,9,5,1

Computed LRU page faults = 12.

Q6(c) Short notes

- Copy-on-write: shared pages copied only on write.
- Thrashing: excessive paging causing low CPU work.
- Global vs local allocation: frame replacement from all processes vs own process frames only
- Belady anomaly: more frames can sometimes cause more faults (e.g., FIFO).
- Demand paging: load page only when first referenced.

Q7(a) Logical to physical mapping (page size=4 bytes)

(Interpreted page table: p0->f5, p1->f6, p2->f1, p3->f2)

Formula: page = LA//4, offset = LA%4, PA = frame*4 + offset

LA=13: page3, off1 -> frame2 -> PA=9

LA=3 : page0, off3 -> frame5 -> PA=23

LA=8 : page2, off0 -> frame1 -> PA=4

LA=6 : page1, off2 -> frame6 -> PA=26

LA=15: page3, off3 -> frame2 -> PA=11

Q7(b)

(i) Compaction: relocates processes to merge scattered holes into larger contiguous free memory.

(ii) Contiguous vs paging:

- External fragmentation: high in contiguous, none in paging.
- Internal fragmentation: lower/variable in contiguous, possible in paging (last page waste).

Q7(c) EMAT

Given:

Memory access time = 100 ns

Page fault service time = 8 ms = 8,000,000 ns

Page fault rate p = 0.1 percent = 0.001

EMAT = (1-p)*100 + p*(8,000,000 + 100)

= 0.999*100 + 0.001*8,000,100

= 99.9 + 8000.1

= 8100 ns approximately (about 8.1 microseconds)

Q8(a) Semaphore synchronization (S1 before S2 before S3)

Initialize sem12=0, sem23=0

P1: S1; signal(sem12)

P2: wait(sem12); S2; signal(sem23)

P3: wait(sem23); S3

Q8(b) Short explanations

- Many-to-one: many user threads mapped to one kernel thread (low overhead, no true parallelism).
- One-to-one: each user thread mapped to one kernel thread (true parallelism, higher overhead)
- Many-to-many: many user threads multiplexed over many kernel threads (flexibility + scalability).
- Microkernel: minimal kernel (IPC, scheduling, low-level memory); services in user space.
- Concurrency vs parallelism: concurrency = overlapping progress; parallelism = simultaneous execution.
- Dual mode: user mode and kernel mode for protection and controlled privileged operations.

B. 2021 EXAM - SECTION B

Q5(a) Why page size is power of 2

- Address split by bit fields is simple and fast.
- Offset extraction is direct bit masking.
- Hardware implementation becomes efficient.

Q5(b) Logical to physical translation with paging

- 1) CPU generates logical address = (page number, offset).
- 2) MMU checks TLB for page->frame mapping.
- 3) On hit: get frame quickly.
- 4) On miss: read page table entry from memory.
- 5) Combine frame + offset to form physical address.

Q5(c) First-fit, Best-fit, Worst-fit placements

Free blocks: 500,100,200,300,600

Process sizes: 418,202,5,112,96

First-fit allocation:

418->500, 202->300, 5->82, 112->200, 96->100

Best-fit allocation:

418->500, 202->300, 5->82, 112->200, 96->98

Worst-fit allocation:

418->600, 202->500, 5->300, 112->298, 96->295

All processes can be allocated; by standard criterion, Best-fit is considered most efficient, though it may create tiny unusable fragments.

Q5(d) Segment-table physical address problem

- The scanned numeric table/logical addresses are partially unreadable.
- Method to solve each item:
 - a) Check offset < segment length.
 - b) If valid: physical = base + offset.
 - c) Else: trap/segmentation fault.

Q6(a) Page fault handling

Same standard steps listed earlier: trap, validate, locate page, choose frame/victim, I/O, page-table update, restart instruction.

Q6(b) Page faults for 4 frames

Reference string:

1,2,3,4,5,1,2,3,4,1,2,3,5,1,2,3

LRU faults = 10
Optimal faults = 7

Q6(c) Inverted and hashed page tables

- Inverted page table: one entry per physical frame, storing (PID, virtual page).
- Hashed page table: hash virtual page number to bucket chain for quick lookup in large space

Q7(a) Effective memory reference time

Given memory reference = 50 ns

(i) Without TLB: one table access + one data access = 100 ns

(ii) With TLB hit ratio $h=0.75$ and TLB lookup=2 ns:

$$\begin{aligned} \text{EAT} &= h \cdot (2+50) + (1-h) \cdot (2+100) \\ &= 0.75 \cdot 52 + 0.25 \cdot 102 \\ &= 64.5 \text{ ns} \end{aligned}$$

Q7(b) RAID0 vs RAID1 vs RAID5

- RAID0: highest performance, no redundancy.
- RAID1: mirroring, high fault tolerance, high space overhead.
- RAID5: good read performance + parity-based fault tolerance with moderate overhead.

Q7(c) Disk scheduling (0-199, start=100, queue 55,57,37,15,80,160,150,36,185)
(Assume initial direction upward for directional methods)

FCFS total movement = 507

SSTF total movement = 255

C-SCAN total movement = 378

LOOK total movement = 255

Q8(a) Process/thread count for given fork/thread code

Final unique processes = 6

Final total threads = 8

(Extra threads created via thread_create calls = 2)

Q8(b) Concurrency without parallelism

Yes, possible on a single-core CPU via interleaving/time-slicing.

Q8(c) Favoring short jobs

- FCFS: weak discrimination.
- RR: moderate discrimination (depends on quantum).
- Multilevel feedback queue: strongest discrimination toward short/interactive jobs.

END OF SOLUTION SET